

Cloud Computing TASK – 2

- 1. In the AWS Management Console, launch two EC2 instances running a simple web server (use Amazon Linux 2 and install httpd or nginx), and verify both are accessible via their public IPs.**

Ans :

- Log in to the AWS Management Console and open the EC2 Dashboard.
- Click Launch instances.
- Enter Name: WebServer-1.
- Select Amazon Linux 2 AMI.
- Choose t2.micro as the instance type.
- Select or create a key pair.
- Under Network settings:
- Select VPC: InstaVibeVPC
- Select Subnet: PublicSubnet-Zomato
- Enable Auto-assign Public IP
- Create or select a Security Group with:
 - SSH (TCP 22) → My IP
 - HTTP (TCP 80) → Anywhere (0.0.0.0/0)
- Click Launch instance.
- Repeat the same process to launch a second instance named WebServer-2.
 - Connect to each instance using SSH:
 - `sudo yum install httpd -y`
 - `sudo systemctl start httpd`
 - `sudo systemctl enable httpd`

- `echo "<h1>Welcome to WebServer-1</h1>" | sudo tee /var/www/html/index.html`
- `echo "<h1>Welcome to WebServer-2</h1>" | sudo tee /var/www/html/index.html`

2. Set up an Application Load Balancer (ALB) in AWS that distributes incoming HTTP traffic across your two EC2 instances, and test by accessing the ALB's DNS name in your browser to see traffic served from both servers.

Ans :

- Log in to the AWS Management Console and open the EC2 Dashboard.
- In the left navigation pane, click Load Balancers → Create Load Balancer.
- Select Application Load Balancer (ALB).
- Enter the following details:
- Name: InstaVibe-ALB
- Scheme: Internet-facing
- IP address type: IPv4
- Under Network mapping:
- Select VPC: InstaVibeVPC
- Select at least two Availability Zones and their public subnets (if you have only one public subnet, create another public subnet in a different AZ for the ALB).
- Create or select a Security Group that allows:
- HTTP (TCP 80) from 0.0.0.0/0
- Create a Target Group:
- Target type: Instances

- Name: WebServer-TG
- Protocol: HTTP
- Port: 80
- VPC: InstaVibeVPC
- Register both EC2 instances (WebServer-1 and WebServer-2) with the target group and click Include as pending below.
- Click Create Target Group.
- Return to the ALB creation page and select WebServer-TG as the default target group.
- Click Create Load Balancer.
- Wait until the ALB status changes to Active and both targets show Healthy in the Target Groups → Targets tab.
 - Copy the DNS name of the Application Load Balancer from the EC2 console.

3. Change the health check settings of your ALB to use a custom path (e.g., /health) and verify that the load balancer only routes traffic to healthy instances.
Hint: Create a /health endpoint on your web server that returns a 200 OK response.

Ans :

- Log in to the AWS Management Console and open the EC2 Dashboard.
- In the left navigation pane, click Target Groups.
- Select your target group (WebServer-TG).
- Go to the Health checks tab and click Edit.
- Change the Health check path from / to /health.
- Keep the protocol as HTTP and the port as Traffic port (80).
- Save the health check settings.

- Connect to WebServer-1 using SSH:
 - `ssh -i your-key.pem ec2-user@<Public-IP>`
 - create the /health endpoint:
 - `echo "OK" | sudo tee /var/www/html/health`
- 4. Simulate a server failure by stopping one of your EC2 instances and observe how the ALB handles requests — describe what happens and how this demonstrates high availability.**

Ans :

- Log in to the AWS Management Console and open the EC2 Dashboard.
- Go to Instances and identify your two web server instances (WebServer-1 and WebServer-2).
- Select one instance (e.g., WebServer-1).
- Click Instance state → Stop instance and confirm the action.
- Wait a few minutes for the instance to stop completely.
- Go to EC2 → Target Groups → Targets and observe that the stopped instance changes from Healthy to Unhealthy.
- Open the Application Load Balancer (ALB) DNS name in your web browser:
- `http://<ALB-DNS-Name>`
- Refresh the page several times. You should see responses only from the remaining healthy instance (WebServer-2).
- Start the stopped instance again by selecting it and clicking Instance state → Start instance.
- Wait until the instance passes the ALB health checks and its status changes back to Healthy.
 - Refresh the ALB DNS page again. The load balancer will resume distributing traffic across both EC2 instances.

5. Compare Application Load Balancer, Network Load Balancer, and Classic Load Balancer in a table with at least 3 points each, focusing on supported protocols, use cases, and features.

Ans :

Feature	Application Load Balancer (ALB)	Network Load Balancer (NLB)	Classic Load Balancer (CLB)
Supported Protocols	Supports HTTP, HTTPS (Layer 7) and provides advanced routing based on URL paths, hostnames, and headers.	Supports TCP, UDP, TLS, and TCP_UDP (Layer 4) for high-performance network traffic.	Supports HTTP, HTTPS, TCP, and SSL (older Layer 4/Layer 7 load balancing).
Main Use Cases	Best for web applications, APIs, microservices, and container-based applications that need intelligent traffic routing.	Best for high-performance applications, gaming, real-time systems, IoT, and applications requiring low latency.	Used for older applications built before ALB and NLB were introduced; suitable for basic load balancing needs.

Feature	Application Load Balancer (ALB)	Network Load Balancer (NLB)	Classic Load Balancer (CLB)
Routing Features	Provides advanced routing such as path-based routing, host-based routing, redirects, and content-based routing.	Provides simple traffic distribution based on IP addresses and ports with extremely fast processing.	Provides basic routing and load distribution without advanced application-level routing.
Performance	Optimized for HTTP/HTTPS traffic with application-level inspection.	Provides very high throughput, ultra-low latency, and handles millions of requests per second.	Lower performance compared to ALB and NLB; designed for legacy workloads.